



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Gerência de Configuração

DevOps

Docker - Introdução

Prof. Sidartha Carvalho

Slides adaptados da Profa. Lana Mesquita



Aulas Docker

1. Docker: Introdução
2. Docker: Imagens e containers
3. Docker: Storage e volumes
4. Docker: Criando imagens e Docker Compose

Docker

- Tecnologia de containers
 - adotada por grandes empresas, como PayPal, Microsoft, Spotify e outras.
- Virtualização
 - VMs
 - Containers
- Desvantagens da VM
 - Cada aplicação necessita de um sistema operacional para poder ser executada.

Desvantagens de VM

- Sistemas operacionais possuem um custo de
 - memória RAM,
 - disco e
 - processamento.
- É necessário uma capacidade mínima para manter as funcionalidades de um sistema operacional.
- Isto aumenta o seu custo de manutenção a cada sistema que tivermos.
- Custo de configuração (liberar portas, instalar alguma biblioteca específica, etc).
- Segurança: manter o sistema de cada máquina virtual sempre atualizado.

VMs

Outros custos de um S.O.:



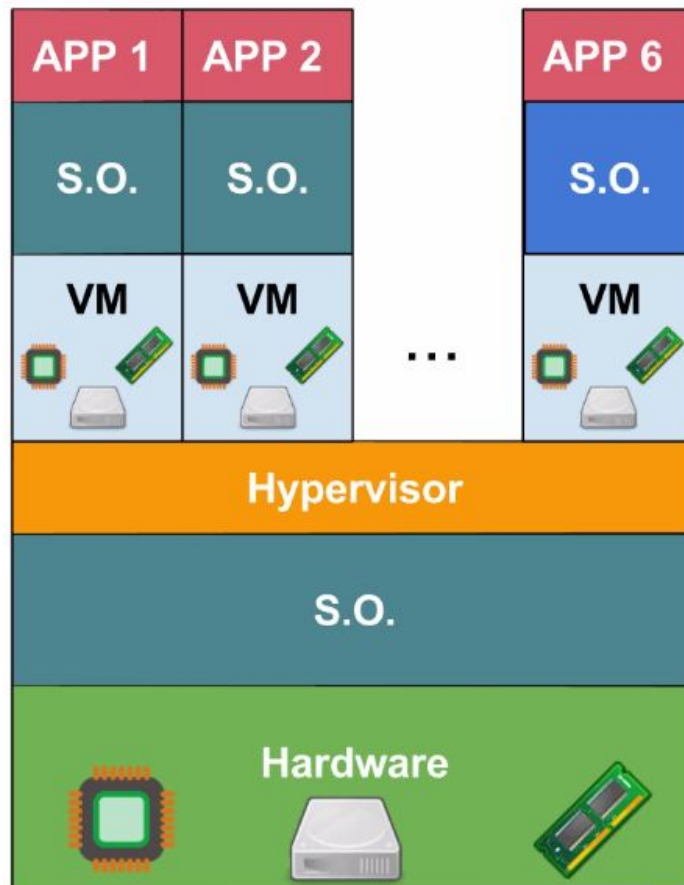
Configuração



Atualização



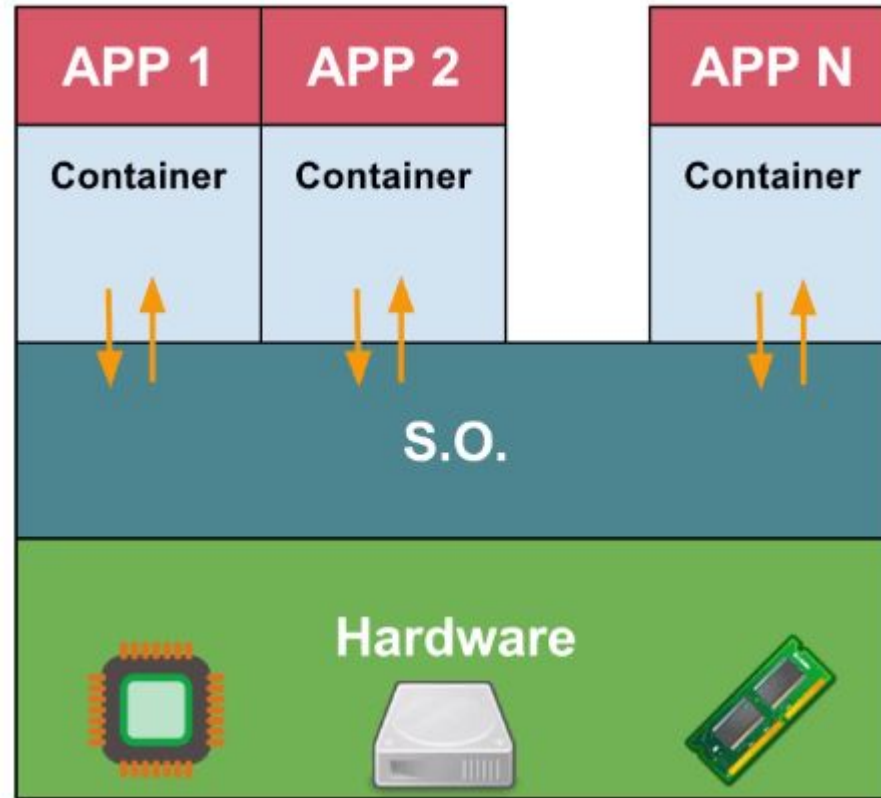
Segurança



Container

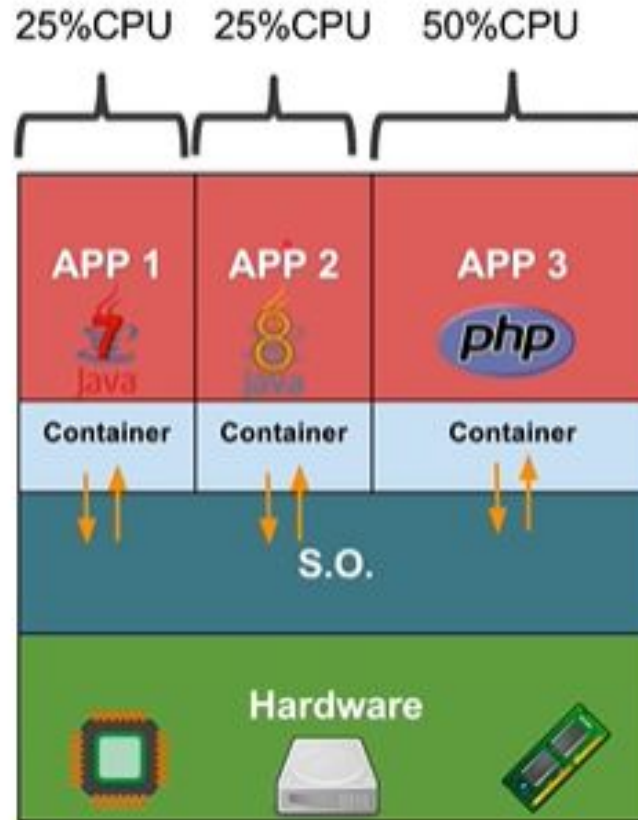
Container

A aplicação é executada dentro do container.



Container

- são mais leves
- não tem custos de manter múltiplos SO
- mais rápido de subir, criar ou remover
- melhor controle dos recursos (CPU, disco, rede...)
- facilidade para trabalhar com diferentes linguagens e bibliotecas



Docker

Docker, Inc.
Empresa



Docker

Software

Docker é um conjunto de produtos de plataforma como serviço que usam virtualização de nível de sistema operacional para entregar software em pacotes chamados contêineres. Os contêineres são isolados uns dos outros e agrupam seus próprios softwares, bibliotecas e arquivos de configuração. [Wikipédia](#)

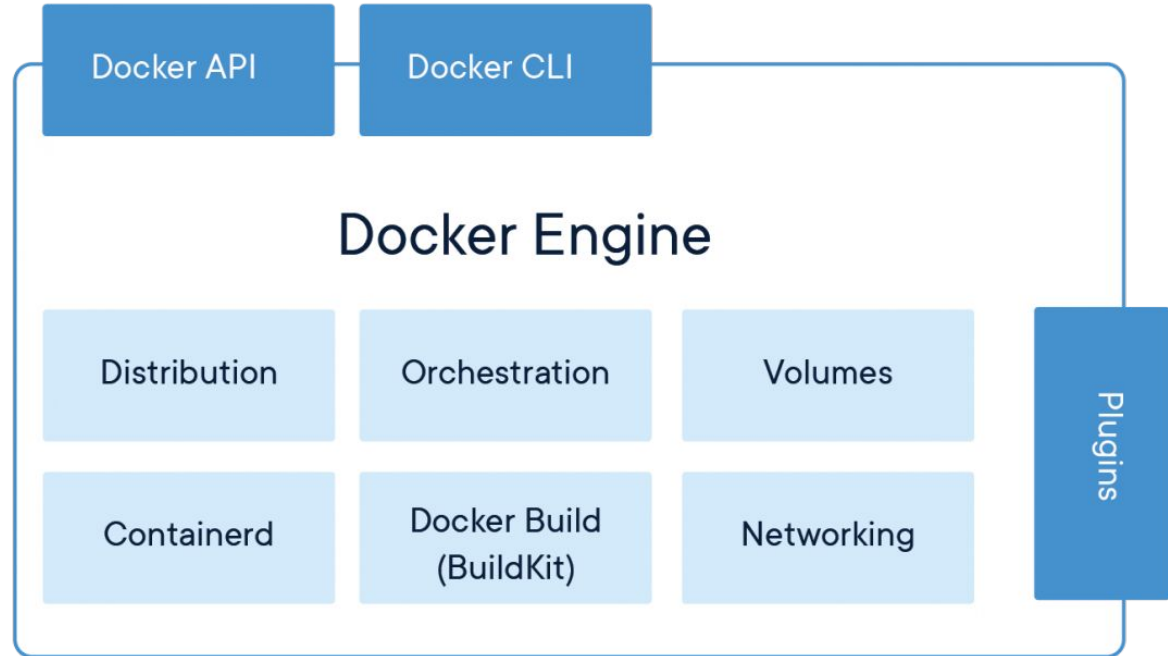
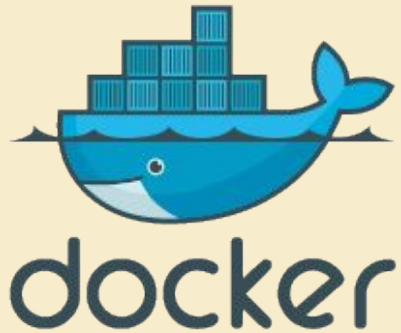
Docker Inc

- **Docker Inc.** (inicialmente dotCloud)
- A dotCloud era uma empresa de **PaaS** (Platform as a Service) responsável pela hospedagem de aplicações
 - levantando o servidor, configurando-o, liberando portas, etc,
- A dotCloud utilizava o Amazon Web Services (AWS), para máquinas virtuais e físicas.
- A dotCloud introduziu o conceito de **containers na hora de subir uma aplicação**, dando origem ao Docker, tecnologia utilizada para baratear o custo de hospedar várias aplicações em uma mesma máquina



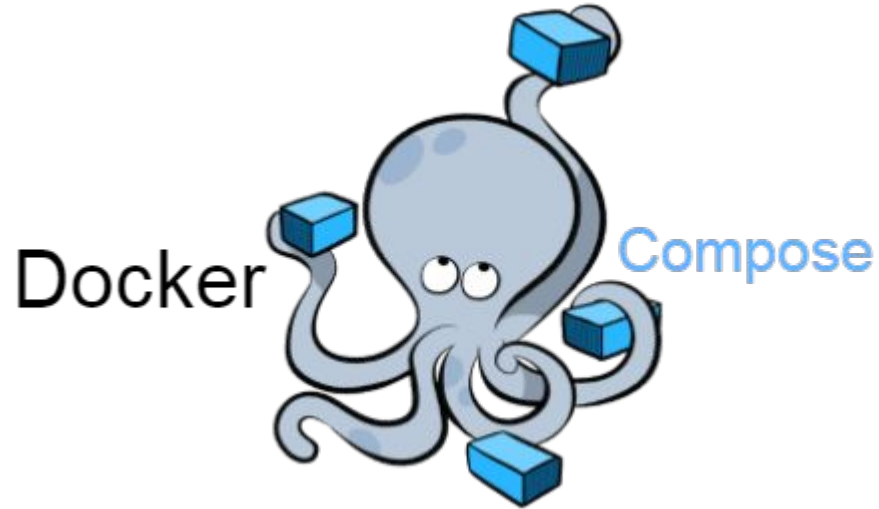
Tecnologias Docker

Docker Engine



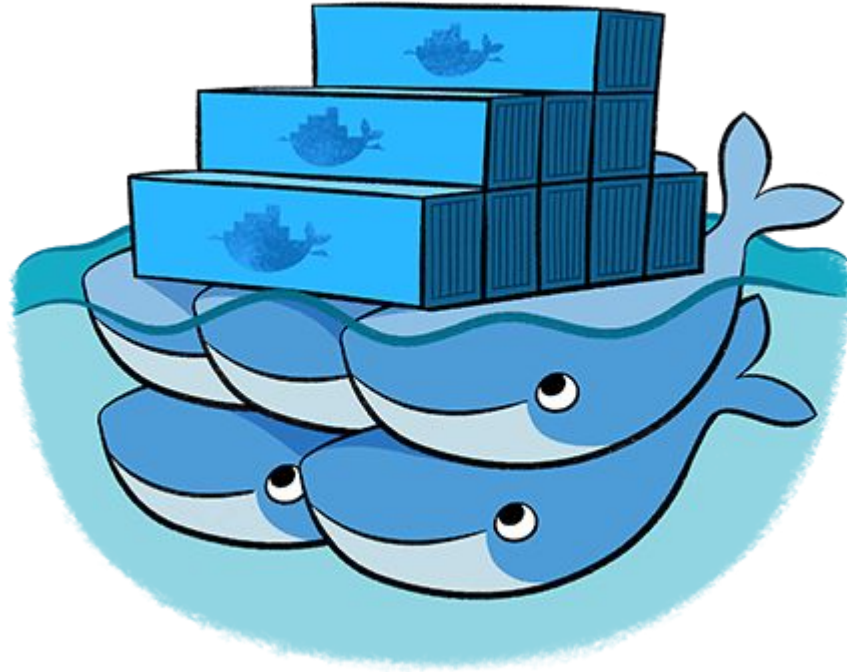
Docker Compose

Orquestrar
containers



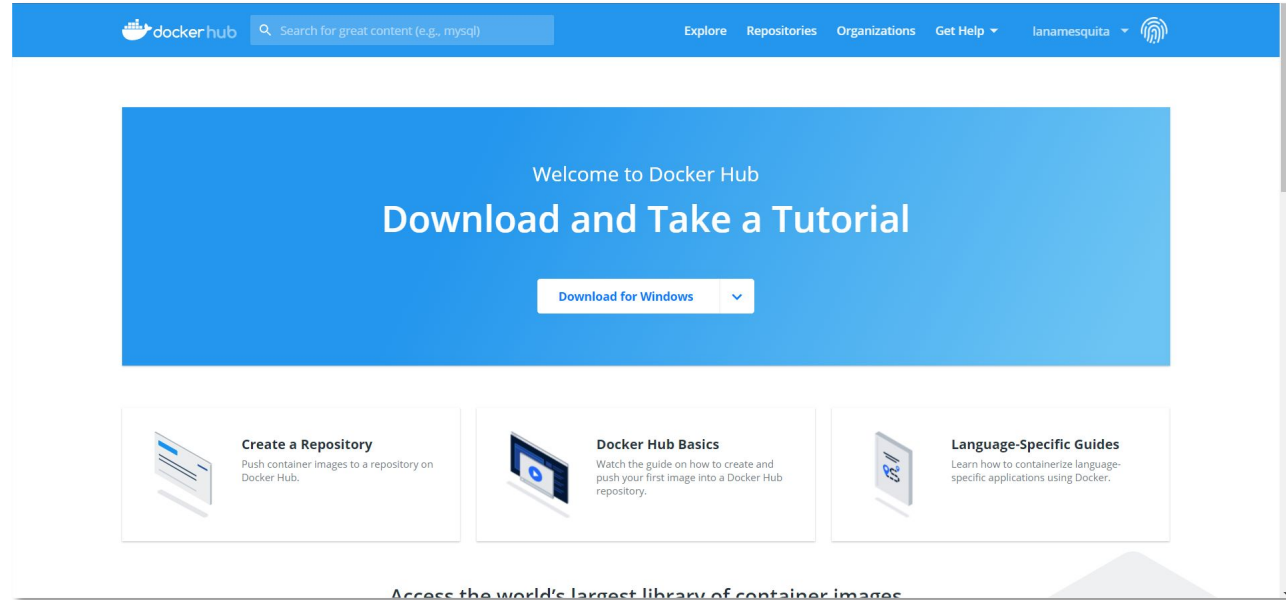
Docker Swarm

Múltiplos Dockers
Host's em cluster



Docker Hub

Repositório com
imagens oficiais e
publicadas



Docker é OpenSource



Docker

Docker provides a simple and powerful developer experience, workflows and collaboration for creating applications.

📍 San Francisco, CA 🔗 <https://www.docker.com> 🐙 @docker ✉ info@docker.com Verified

📁 Repositories 80 📦 Packages 👤 People 14 📁 Projects 2

Pinned repositories



[compose](#)

Define and run multi-container applications with Docker

Python 22.1k 3.7k



[roadmap](#)

Welcome to the Public Roadmap for All Things Docker! We welcome your ideas.

582 78



[awesome-compose](#)

Awesome Docker Compose samples

HTML 6.5k 778



[build-push-action](#)

GitHub Action to build and push Docker images with Buildx

TypeScript 1.2k 126



[compose-cli](#)

Easily run your Compose application to the cloud with compose-cli

Go 347 82



[node-sdk](#)

Docker CLI gRPC JavaScript SDK

JavaScript 35 19

🔍 Find a repository...

Type ▾

Language ▾

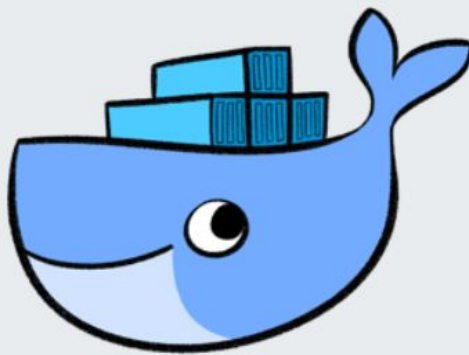
Sort ▾

Instalação

Instalação

- Linux:
 - <https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-on-ubuntu-20-04-pt>
- Windows:
 - <https://docs.docker.com/docker-for-windows/install/>
 - Vejam os pré requisitos
 - Arquitetura 64 bits
 - Versão Pro, Enterprise ou Education.
 - Virtualização habilitada

Play with Docker



Play with Docker

A simple, interactive and fun playground to learn Docker

Login ▼

Comandos básicos para testar

- Visualizando a máquina hospedeira do Docker
 - `cat /etc/*release*`
 - `lscpu`
 - `free -b`
- `docker run hello-world`
- `docker version`

03:50:05

CLOSE SESSION

Instances



+ ADD NEW INSTANCE

192.168.0.8
node1

GIVE FEEDBACK

ctg66ggl_ctg66l0l2o90008fm790

IP

192.168.0.8

OPEN PORT

Memory

3.49% (139.7MiB / 3.906GiB)

CPU

0.13%

SSH

ssh ip172-18-0-19-ctg66ggl2o90008fm78g@direct.labs.play



DELETE

 EDITOR

Caches (sum of all):

```
L1d:      128 KiB (4 instances)
L1i:      128 KiB (4 instances)
L2:       1 MiB (4 instances)
L3:      50 MiB (1 instance)
```

NUMA:

```
NUMA node(s):      1
NUMA node0 CPU(s): 0-7
```

Vulnerabilities:

```
Itlb multihit:      KVM: Mitigation: Split huge pages
L1tf:               Mitigation; PTE Inversion; VMX conditional cache flushes, SMT vulnerable
Mds:               Mitigation; Clear CPU buffers; SMT Host state unknown
Meltdown:          Mitigation; PTI
Spec store bypass:  Vulnerable
Spectre v1:         Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:         Mitigation; Full generic retpoline, IBPB kernel-off, STIBP disabled, RSB filling
Srbds:             Not affected
Max-amount-affected: Mitigation; Clear CPU buffers; SMT Host state unknown
```



```
Docker Playground x +
labs.play-with-docker.com/p/c1adn3oh550g00fnrmo0#c1adn3oh_c1adtr0h550g00fnrmdg
[node1] (local) root@192.168.0.18 ~
$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

[node1] (local) root@192.168.0.18 ~
$ docker version

Client: Docker Engine - Community
 Version:           20.10.0
 API version:       1.41
 Go version:        go1.13.15
 Git commit:        7287ab3
 Built:             Tue Dec  8 18:54:00 2020
 OS/Arch:           linux/amd64
 Context:           default
 Experimental:      true

Server: Docker Engine - Community
 Engine:
  Version:          20.10.0
```

Dica: alt + enter

Dica:
Copiar: ctrl + insert
Colar: shift +
insert

Referências

- [Curso Docker Alura](#)
- [Curso Introdução ao Docker UFC / Insight Lab](#)
- [Documentação Docker](#)

Mostrar os exemplos do próprio Docker Desktop

[← Learning center](#)

×

UNIVERSIDADE
FEDERAL do CEARÁ
MARACÁ - CAMPUS QUIXADÁ

Walkthroughs



Multi-container applications
4 of 9 completed

`$ docker init`

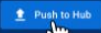
Containerize your application
3 mins



Persist your data between containers
3 mins



Access your local folder from a container
6 mins



Publish your image
5 mins

```
1 FROM node
2 RUN mkdir -p
3 WORKDIR /app
4 COPY package
```

How do I run a container?
✓ Completed



What is a container?
✓ Completed



Run Docker Hub images
✓ Completed

Dúvidas?